

.NET 11.0 E2E test scenarios

Tuesday, November 18, 2025 3:14 PM

Windows VM on DevBox - Win 11 25H2

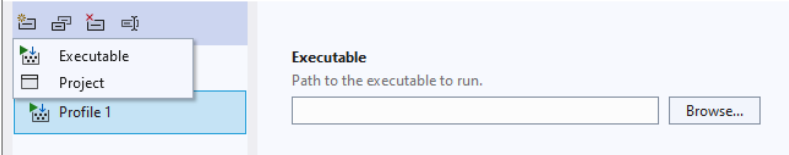
Arm64 VM on Azure

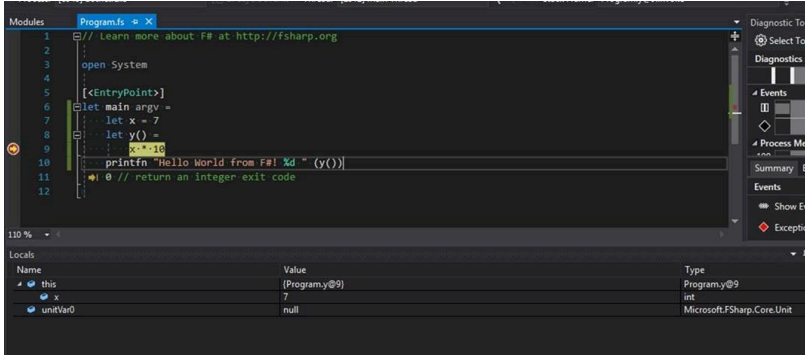
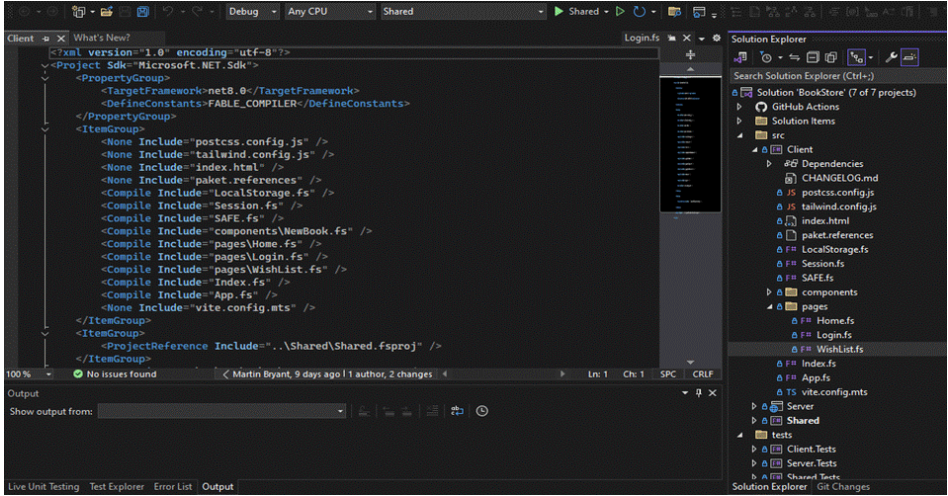
- Win11 (24h2): Windows-11-Enterprise-24H2-ARM64
- Winserver2022: Windows-Server-2022-ARM64
- Winserver 2025: ge_prerelease-vhdx_server_serverdatacenter_en-us_vl-arm64

#	Priority	VS or CLI	Brief Description	Detailed Scenario
				SDK installation
1	P0	Installer	SDK with WiX signing	- Create a windows VM - Copy the CIP file from \\bvtst2\team\2eshare\allowEnforceSignCI to a folder  allowEnforceSignCI - Open an admin prompt and run <code>citool --update-policy allowEnforceSignCI.cip</code> in above folder - Restart the machine citool --list-policies <pre> .\Windows\System32>citool --list-policies Policy ID: a244370e-44c9-4c06-b551-f6016e563076 Base Policy ID: a244370e-44c9-4c06-b551-f6016e563076 Friendly Name: AllowMicrosoft_2024-05-17 Version: 2814754062139393 Platform Policy: false Has File on Disk: true Is Currently Enforced: true Is Authorized: true Status: 0 </pre> - Try to run the .NET installation bundles (EXE). If it was properly signed the install should succeed. Note: - Only candidate builds for upcoming servicing releases and official previews will be signed properly to test this - Known issue: 41010: [NETSDKE2E]Running project failed with net9 preview 4 sdk after allowEnforceSignCI policy enable it would block running locally built applications because they are unsigned. Please follow below steps to remove citool --remove-policy {a244370e-44c9-4c06-b551-f6016e563076} and then restart - please create a win11 to cover it as citool not support on win10 VM info: 172.16.60.195 DESKTOP-912OPEA\2e SDK Path: \\DESKTOP-912OPEA\c\$\SDKTesting
2	P0	Installer	SDK upgrade in place	- Install the previous version of the SDK band. For .Net 11, install the previous release of the .NET 11 SDK - Install the SDK to test. - Verify that the SDK from step 1. is not installed any more. - Verify that basic scenarios work after the upgrade.
3	P1	Installer	Install SDK in a custom location	<i>Note: Use this kind of installation few times in a week and not daily.</i> Preconditions: Visual Studio is not installed There are no SDKs on the machine The file: %ProgramFiles%\dotnet\dotnet.exe does not exist The file: %ProgramFiles(x86)%\dotnet\dotnet.exe does not exist The following reg keys do not exist: HKEY_LOCAL_MACHINE\SOFTWARE\WOW6432Node\dotnet\Setup\InstalledVersions\x64\sdks\InstallLocation HKEY_LOCAL_MACHINE\SOFTWARE\WOW6432Node\dotnet\Setup\InstalledVersions\x86\sdks\InstallLocation Note: All the above outlined prerequisites are to ensure that a previously installed SDK does not interfere with testing. Install the SDK in a drive different than the drive containing Visual Studio installation. This means If VS is installed in C:\ then install the SDK in D:\ or E:\ Example of how to use 'installpath' option to specify the path - dotnet-sdk-3.0.100-alpha1-009719-win-x64.exe /install DOTNETHOME_X64="D:\dotnet\x64" DOTNETHOME_X86="D:\dotne /norestart *case matters for the parameters: "DOTNETHOME_X64" and "DOTNETHOME_X86".

4	P1	Installer	Verify the version number mentioned on the installation wizard	The wizard should mention the version number of the SDK you are testing. - Notice the version number - Notice that 'preview' is mentioned only when the SDK is a preview Microsoft .NET SDK 8.0.100-preview.1.23115.2 .NET SDK The .NET SDK is used to build, run, and test .NET applications. You can choose from multiple languages, editors, and developer tools, and take advantage of a large ecosystem of libraries to build apps for web, mobile, desktop, gaming, and IoT. We hope you enjoy it! If you plan to use .NET 8.0 with Visual Studio, Visual Studio 2022 17.4 or newer is required. Learn more. .NET By clicking Install, you agree to the following terms: Privacy Statement Telemetry collection and opt-out Licensing Information for .NET Install Close Microsoft .NET SDK 8.0.100-preview.1.23115.2 Setup Progress Processing: Microsoft .NET Runtime - 8.0.0 Preview 1 (x64) .NET Cancel Offline scenarios			
5	P1	Testing when machine is offline <table><tr><td>SDK</td><td>Offline mode</td></tr><tr><td>11.x.xx</td><td>Only need to cover 11.0 offline scenario,</td></tr></table>	SDK	Offline mode	11.x.xx	Only need to cover 11.0 offline scenario,	Validate that scenarios below function as expected when the machine is disconnected from the internet. Ensure that test online to ensure the machine state is not polluted by a previously tested online scenario. All scenarios do not need to be rotate through them all on some regular cadence. Note that F#, MSTest and Xunit are not part of the offline feed. These are 2 scenarios you should test - 1. Create, build, run, publish a C# console application referencing a Netstandard class library. Ensure that an API is invoked such as string is passed between them 2. Create, build, run, publish a WPF application Workaround for testing offline scenarios(done on a clean machine): 3. Use 'dotnet new nugetconfig' to generate a NuGet.config 4. Delete the example source for "b" so that the file only has this content: <pre><?xml version="1.0" encoding="utf-8"?> <configuration> <packageSources> <!--To inherit the global NuGet package sources remove the <clear/> line below --> <clear /> </packageSources> </configuration></pre> 5. With this file in place, run 'dotnet new console', then the error still occurs.
SDK	Offline mode						
11.x.xx	Only need to cover 11.0 offline scenario,						
				.Net 11.x Preview version			
7.NET Core 3.0 test scenarios with VSCode on macOS	P2	VS	Notify user if he/she is not using compatible versions of VS and .NET Core SDK	.NET Core SDKs compatibility with Visual Studio versions If you are testing .Net 11.0 SDK - .NET 11.0 is compatible with VS (most likely 18.x and higher though 11.0 Preview 1 is working with 18.1 as well). When opening an existing 11.0 project on 17.14 or lower, a warning with similar message as below will show up - Output Show output from: Build Rebuild started at 1:26 AM... ----- Rebuild all started: Project: myconsole10, Configuration: Debug Any CPU ----- Restored C:\tt\myconsole10\myconsole10.csproj (in 2 ms). [net] Program files\dotnet\sdk\8.0.100\sdks\Microsoft.NET.Sdk.DefaultItems.targets(145,3): warning NETSDK1233: Targeting .NET 10.0 or higher in Visual Studio 2022 17.14 is not supported. myconsole10 -> C:\tt\myconsole10\bin\Debug\net10.0\myconsole10.dll Done building project "myconsole10.csproj". ***** Rebuild All: 1 succeeded, 0 failed, 0 skipped ***** ***** Rebuild completed at 1:26 AM and took 01.060 seconds *****			

				.Net 11.x final (stabilized) version
8	P0	CLI , VS	Ensure that Correct versions of runtime and SDKs are getting installed	Test this only if you are testing final version of .NET 11.x - Whenever a final (stabilized) version of .NET 11.x is being tested, either as a standalone SDK or as part of a Visual Studio preview , run 'dotnet --info' to ensure the .NET Core Runtimes and SDKs installed do not have build suffixes such as "-se numbers of the SDKs and runtimes should consist only of "<major>.<minor>.<patch>" (e.g. '11.0.0').
				Newer SDKs should also get patches for older runtimes
9	P0/P1	CLI	Older run times should also be patched in the SDKs	Suppose there is an update of .NET Core x.y.z (e.g. 2.1.9) For each newer SDK shipping in this update that is >= x.y (say 2.1.109, 2.2.104, etc.), test this: • dotnet new global.json • edit global.json and set the SDK version to test (again, say 2.1.109, then repeat for 2.2.104, etc.) • mkdir test • cd test • dotnet new console • edit test.csproj and set TargetFramework=netcoreappx.y (netcoreapp2.1 in our example) • dotnet publish -r win-x86 --self-contained • check the contents of bin\Release\net<x.y>\win-x64\publish\test.deps.json, search for "Microsoft.NETCore.App/" and msrc. For example: "'Microsoft.NETCore.App/2.1.9'": <i>Note: Please make sure that you have included the feed for this older runtime (2.1.x in the above case) This scenario only need to be covered in monthly update</i>
				Template test
10	P0	VS	Template test	When creating a new project for the first time in a given VS instance, we will default to the latest .NET Core LTS release. If the template doesn't support any LTS releases, we will fallback to the highest .NET Core release, favoring non-preview releases over current Console and Class Library experience.
				Create new solutions with .NET projects
11	P0	VS	Single TFM (VS)	Create a .Net console app. Build and run. Check what fx it is targeting. In the test results report, for this test case, please specify targeting. If you feel the target fx does not seem right, please report it as a bug.
12	P0	CLI	Single TFM (CLI)	Using CLI : Create an empty solution, add a .NET Console application with reference to .NET / .NET Standard class library, build and run. Ensure that an API is invoked from the library and an argument such as string is passed between them
13	P0	CLI	Multi TFM scenario for C# (CLI)	Using CLI: Create an empty solution, add a C# Console application which targets net48 and net11.x. Build and Run for each target framework. • Modify csproj like this <pre><Project Sdk="Microsoft.NET.Sdk"> <PropertyGroup> <OutputType>Exe</OutputType> <TargetFrameworks>net11.0;net48</TargetFrameworks> <ImplicitUsings>disable</ImplicitUsings> <Nullable>enable</Nullable> <LangVersion>latest</LangVersion> </PropertyGroup> </Project></pre> • Add 'using System;' in Program.cs file.
14	P0	VS	Multi TFM scenario for C# (VS)	Create new .NET 11.x console application. Modify like below, then build and run for each target framework. • Modify csproj like this <pre><Project Sdk="Microsoft.NET.Sdk"> <PropertyGroup> <OutputType>Exe</OutputType> <TargetFrameworks>net11.0;net48</TargetFrameworks> <ImplicitUsings>disable</ImplicitUsings> <Nullable>enable</Nullable> <LangVersion>latest</LangVersion> </PropertyGroup> </Project></pre> • Add 'using System;' in Program.cs file.
				Retargeting
15	P2	VS	Retargeting to lower version	Create new .NET 11.0 console application and change it to target a lower version build and run. As examples If you are testing 11.0 SDK, start with a 11.0 project and then change the target fx to 10.0, 9.0, 8.0, Build and run in all configurations
				Referencing
16	P1	VS	17.a.NET Standard - NET Framework Interop: NET Standard 2.x compliant platforms can reference	In Visual Studio, create one of several .NET /NET Standard 2.x projects and reference projects, packages and binaries that target .NET Standard 2.0. For a detailed walkthrough at: NetCoreApp To Net472 Walkthroughs

			.NET framework libraries (up to 4.6.1)	
17	P0	VS	17.b.NET Standard - .NET Framework Interop: .NET Framework (4.6.1 and above) can reference .NET Standard 2.x	In Visual Studio, create one of several .NET Framework projects and reference projects, packages and binaries that target .NET detailed walkthrough at: Net472 to NetStandard Walkthroughs
18	P2	VS	19.Package downgrades occurring due to "nearest wins", are errors by default	Create a .NET / .NET Standard project and verify that package references that trigger package downgrade diagnostics report t walkthrough at: Package Downgrades as Errors
19	P2	VS	17.c.NET Standard - .NET Framework Interop: Unsupported P2P and assembly references show adequate errors 32.Error and general output message improvements	Create a .NET / .NET Standard project and simulate several NuGet error scenarios. Detailed walkthrough at: NuGet Error Scen
20	P2	VS	Changes in new profile should be reflected in LaunchSettings.json	Create a console app, from project properties -> debug tab, Open debug launch profiles UI, click 'create a new profile' button  For this profile, there will be a launchSettings.json file created in the project. Any changes you make to the fields in the debug reflected in this json file. You should be able to build and run your project in this profile by providing the required values.
				Unit Testing and LUT
21	P1	VS	Live Unit Testing for .NET Projects	1.Create a solution with .Net standard 2.x class lib and .Net xunit project. Verify that LUT works in this solution
22	P0	VS	Test Explorer in .NET Projects	Create a .NetStandard Class library with a referenced test project(Test all the 3 test frameworks - Xunit, Nunit, Mstest). Add test. 1.Ensure that Test Discovery automatically triggers. 2. Ensure that Run All/ Run from context and Run Selected tests work. Perform the same for Debug. 1.Ensure that Test Indicators show up in the editor on build.
				VB
23	P0	VS		Create a VB .NET Console application which references C# .NET / .NET Standard class library, build and run
				F#
24	P0	CLI	F# Support for .NET projects	Using CLI : Create an empty solution, add a F# Console [net11.0] application which references F# .NET / .NET Standard [2.x] c invoked from the library and an argument such as string is passed between them build and run Code: <pre>// Program.fs // For more information see https://aka.ms/fsharp-console-apps printfn "Hello from F#" lib1.Say1.hello "Kevin" // lib1.fs namespace lib1 open lib2 module Say1 = let hello name = Say2.hello name // lib2.fs namespace lib2 module Say2 = let hello name = printfn "Hello %s" name</pre>
25	P0	VS	UT in F#	Using VS- Create an F# console app that references F# .NET/ .NET Standard class lib. Build and run. Add a C# .Net Unit Test project or xUnit Test project, verify that LUT works with the C# .Net Unit Test project or xUnit Test pr Note: LUT doesn't support F#.

26	P1	VS		Debugging in F# - - At the command line type: - Dotnet new console -lang F# - Edit Program.fs – replace the contents of the file with this: <pre>// Learn more about F# at http://fsharp.org open System [<EntryPoint>] let main argv = let x=7 let y()= x*10 printfn "Hello World from F#! %d" (y()) 0 // return an integer exit code</pre> - Build it using: dotnet build - Open project on VS - Place a breakpoint on the line: x * 10 - Press F5 x*10 should be highlighted with a yellow bar - Open locals windows, open this and observe 
27	P1	CLI		Using CLI on windows pc with VS: Create an empty solution, add a F# Console application which targets net48. Build and Run Check with Phillip
28	P1	CLI	Multi TFM scenario for F#	Using CLI on windows pc with VS: Create an empty solution, add a F# Console application which targets net48 and net11.0. framework.
30	P1	VS		Fsharp Tree Ordering (Use clone of https://github.com/SAFE-Stack/SAFE-BookStore for this step) - Open the solution at the root in Visual Studio. For each project in the solution, verify that file order in Solution Explorer is project file. (i.e. instead of alphabetical order): For current 8.0  - LocalStorage.fs - Session.fs - SAFE.fs - components\NewBook.fs - pages\Home.fs - pages\Login.fs - pages\WishList.fs

				- Shared.fs - Index.fs - App.fs (Note: Tree order is determined when solution is loaded. Any new files are added to the top of the tree.)
31	P0	VS		F# and ASP.NET projects - In Visual Studio, ensure that you have the ASP.NET workload installed. - File New Project F# ASP.NET web app Web API. - Ensure that the .NET dropdown and Docker checkbox are there. - After the project loads, add a new F# file. - Ensure that it is added and ordered at the top of the file list. - Debug the application. - Ensure that it runs. Note: Change from "launchUrl": "api/values" to "launchUrl": "weatherforecast" in CLI
				Debugging
32	P0	VS	Basic debugging	- Create new .NET 11.x console application with reference to a .NET / .NET Standard class library - build and run. Default ver - Debug the app - Put breakpoint in the code. F5 and make sure the breakpoint is hit. - Create new .NET 11.x console application with reference to a .NET Standard 2.1 class library - build and run.
33	P0	VS	'IL symbols' (used by the debugger)	Use steps for testing on Windows from this page - IL symbols scenarios <i>Note: When first step into the concatenation method (F11), it shows the source code for ToString(), not the source code for Console behavior looks by design. This is a change in VS Master branch for both full fx and .NET.</i>
34	P1	VS	'Native Symbols' (used by the profiler)	- Download PerfView from https://github.com/Microsoft/perfview/blob/master/documentation/Downloading.md (this or - Start data collection by running. It will ask for elevation (grant it) It should bring up a dialog box that shows that it is col PerfView /merge:false collect - In the razor directory you built previously (in the above scenario # 30p0) run the application by running <pre>dotnet bin\Debug\net11.0\razor.dll</pre> It should give some text and the indicate that to stop it Press Control C. Go ahead and press Control C. - Stop PerfView collection by hitting the 'stop collection' button in PerfView. - It will have created a 'PerfView.etl' file that shows in the PerfView's window. Double click on it to open - Double click on the 'CPU Stacks' view this brings up a dialog box with every process running during the collection. - Select the 'dotnet' executable that was running 'razor.dll' (The program args are there if needed). This brings up a - The view will have nodes that look like <microsoft.aspnetcore.hosting!>> that have ? in them (meaning it does not <microsoft.aspnetcore.hosting!>> and press Alt-S (lookup symbols). Do this for the following DLLs <pre>microsoft.aspnetcore.hosting system.private.corelib system.linq</pre> Each time you hit 'Alt-S' it should resolve the symbols (it may seem like it disappeared because that one node tur in sorted order). If the node does NOT change, then that is an error report it. For each of these find one of the resolved nodes (you can use the 'Find' box at the top to search for it). Select the complete method name in it) and type Alt-D (lookup definition). It SHOULD open a source code window. If it does not t Note, in case of failure you can collect the log like this attach to the bug - In general PerfView has a 'log' (button in the lower right corner of every window), and does log quite a bit of detailed info and source lookup. This is useful in determining what went wrong. When a failure occurs, click on the 'Log' button, then hit 'Ctrl-A' (which selects all text) 'Ctrl-C' Copy it), then open Notepad; that if the log is very long, PerfView truncates it but mentioned at the top the file name where you can find the complete
				Open existing solutions with .NET projects
35	P1	VS	Current 11.x SDK can build projects targeting lower SDK versions	If you are testing 11.x SDK - - Open an existing .NET Core 10.0 console application with reference to a .NET Standard class lib. Build and run - Open an existing .NET Core 9.0 console application with reference to a .NET Standard class lib. Build and run - Open an existing .NET Core 8.0 console application with reference to a .NET Standard class lib. Build and run
36	P1	CLI	Make sure new SDK can build and run the lowest patch version of the last stable release	Make sure to not install VS for this. Install SDK only If you are testing 11.0.x SDK for which 10.0 is the latest stable release - Install 10.0.100 SDK (runtime 10.0.0 which is lowest patch), create app targeting 10.0. Build and run. Install 11.0.x SDK. using the new SDK
37	P1	VS	Projects targeting lower SDK versions, can be re-targeted to current 11.x SDK	If you are testing 11.x SDK - - Open an existing .NET Core 10.0 console application with reference to a .NET Standard class lib. Retarget it to 11.x, build and run - Open an existing .NET Core 9.0 console application with reference to a .NET Standard class lib. Retarget it to 11.x, build and run - Open an existing .NET Core 8.0 console application with reference to a .NET Standard class lib. Retarget it to 11.x, build and run

				- Open an existing multi Tfm project <TargetFrameworks>net48; net8.0</TargetFrameworks>. Retarget it to 11.x (<TargetFramework>net11.0</TargetFramework>) "\\bvtsrv2\team\eshare\ConsoleApp80-MultiTfmnet8net48.zip"
				Set up Scenarios
38	P0	VS	Full suite of set up scenarios	Run few of these on each version of SDK you are testing. If you can run all of them then that is ideal. .NET F2E Setup scenarios
				SxS install of Stable and Preview
39	P1	VS	Released stable VS + VS under test, function well when installed SxS	Install latest stable Dev18 . Install VS version to test. If you are testing SDK 11.x - - Verify that .NET 10.x C# console app can be created correctly in VS version to test - build and run. - Verify that .NET 9.x C# console app can be created correctly in VS version to test - build and run. - Verify that .NET 8.x C# console app can be created correctly in VS version to test - build and run.
40	P1	VS		Install latest stable release of Dev18. Install CLI/SDK 11.x. Now uninstall CLI/SDK 11.x. Verify that projects can be created in t and that the projects can build and run
				VS upgrade
42	P1	VS	VS upgrade	1. Install the VS version that contains the previous SDK version. 2. Upgrade the VS version above to the one containing the SDK under test. 3. Run scenarios to make sure everything works well.
				Other scenarios
43	P1	VS	SDK Resolver using Global.json	The resolver side of the changes are pretty much covered by simply using VS for .NET . However, I added one manual coverage for it: - Pin the SDK version through global.json - Create a new 11.x console app in VS in a new solution - Edit the csproj and make it target 10.0: change <TargetFramework>net11.x</TargetFramework> to <TargetFramework>net10.0</TargetFramework>. - Add a global.json file in your solution folder with the following content. Be careful to replace the version placeho that is being installed with VS: <pre>{ "sdk": { "version": "<the_cli_version_that_comes_with_vs>" }</pre> - Run your project. - It should run successfully.
44	P2 Loweri ng priority since it is covere d in automa tion	CLI	Self contained apps	- Find an appropriate RuntimeIdentifier (RID) for the current OS here: https://docs.microsoft.com/en-us/dotnet/core/rid-c "dotnet new console" "dotnet publish -r <RID> --self-contained" "cd bin\Release\<targetframework>\<RID>\publish" - Replace backslashes with slashes on non-Windows OS's - Run the app executable - On Windows, this will be <appname>.exe, while on Linux/OS X it will just be <appname> - The app should print "Hello World!" <i>Note:</i> The default for -c, --configuration option should be 'Release' for NET 8.0 projects and above, but 'Debug' for older projects [NETSDKE2E]With .NET 8 SDK installed, the default publish path is "/bin/Release/..." not "/bin/Debug/..." when running "d
45	P0/P1/ p2	CLI	Fx dependent publishing. (Only for 2.1.4xx and higher SDKs)	Should be run for .Net Core 2.2 and above - Scenarios are here - Framework-Depende nt Publishing with Application Host . P scenarios and run it. Please run P0s when doing P0 test pass, P1s when doing a P1 test pass and P2s when doing a P2 test pass from the above doc
46	P1	CLI	Framework-dependent apphost by default	Through CLI - Check on Windows and on any one flavor of Linux - Create a .NET console application: 'dotnet new console'. - Run 'dotnet build'. - Check the contents of the 'bin/Debug/net11.0' sub-directory. - On Windows, a '<project_name>.exe' file should be present. - On non-Windows platforms, a '<project_name>' file should be present that is +x (executable). - Run the apphost directly (e.g. 'bin/Debug/net11.0/<project_name>.exe'). "Hello world" should be printed. - Add 'System.Threading.Thread.Sleep(3 * 60 * 1000);' to 'Main' in 'Program.cs'. - Run 'dotnet run' to run the application. - Use Task Manager (Windows) or use 'ps aux grep <project_name>' to ensure that the apphost is running. 'dotnet exec' present. - Ctrl-C the running 'dotnet run' command.
47	P1	VS	Apphost customization	Through VS - Note: Windows Only - Create a WPF application: 'dotnet new wpf'. - Run 'dotnet build'. - In Explorer, navigate to the 'bin/Debug/net 11.0' sub-directory.

				- Right click the `<code><project_name>.exe</code>` file and select `Properties` and then the `Details` tab. - Ensure the `file version` is `1.0.0.0`. - Edit `<code><project_name>.csproj</code>` and set the following property: `<code><AssemblyVersion>2.0.0.0</AssemblyVersion></code>`. - Run `dotnet build`. - Repeat steps 3 and 4 to view the apphost file properties. - Ensure the `file version` is now `2.0.0.0`. - Create or copy an existing icon file to the project's directory as `app.ico`. - Edit `<code><project_name>.csproj</code>` and set the following property: `<code><ApplicationIcon>app.ico</ApplicationIcon></code>`. - Run `dotnet build`. - View the `<code><project_name>.exe</code>` file in Explorer again; the application icon should match `app.ico`. - Double click `<code><project_name>.exe</code>`. The default WPF application should run without any console window created. - Run `dotnet run`. Ensure `<code><project_name>.exe</code>` is running in Task Manager.
48	P0	CLI	Installers for all supported Linux distro	Use native installers for the distros on which we have native installers: Native installers. For some distros install using zip installers for others install using native installers.
49	P2		Bundle CLI in the Build Tools SKU for Dev18	- Install the Build Tools SKU (installer for it under bootstrappers\BuildTools) - Install the Web Tool Bundle - Create an mvc app through the dotnet command - Run the mvc app through the dotnet command
50	P2	CLI/VS	Localization	Run the below command in the CLI and verify that messages shown as a result are all localized - Dotnet build --help With VS, no scenario of its own, but some days we should run through all the scenarios with Windows and VS being non-Engl characters(covering both typing with local keyboard and copy/paste) into Project name/Class name/Method name/publis With CLIs, run the below command in the CLI and verify that messages shown as a result are all localized - Dotnet build --help
Common 51-55	P1	VS	Common setup for 48-51	Scenario 2: NuGetPackageExplorer [Windows] [Framework Dependent] - Clone NuGetPackageExplorer Repo - Fot .net 10: git clone https://github.com/NuGetPackageExplorer/NuGetPackageExplorer - Fot .net 8: <code>\\bvtsrv2\team\eshare\NPE\8.0</code> project - Fot .net 9: <code>\\bvtsrv2\team\eshare\NPE\9.0</code> project - Build the solution in CLI - dotnet publish -c Release -r win-x64 /p:PublishReadyToRun=true /p:SelfContained=true NuGetPackageExplorer.c - Launch bin\Release\<targetframework>\win-x64\publish\NuGetPackageExplorer.exe, go to "Open a package fro any errors. - Open NuGetPackageExplorer.sln in VS 2026 - Open Configuration manager and set: - Configuration to Release - Set Platform to x86 (add the configuration). - Publish the NuGetPackageExplorer.csproj project as a framework-dependent app - Create a new FolderProfile - Edit the settings to the following (if not chosen by default) - Configuration = Release X86 - Target Framework = net10.0 - Deploy mode = Framework Dependent - Target Runtime = Portable - Publish - Run the published NuGetPackageExplorer.exe(Do this under the specific published folder) and try out a few options. Scenario 3: HelloWorld [cross-platform] On a windows machine, do: - mkdir hello - dotnet new console - dotnet publish -c Release -r osx-x64 --self-contained Take the Contents of the publish directory to a mac machine ((Mac machine doesn't need the SDK installed).) - Run the binary: hello - Expect to see "Hello World!"
54	P1	VS and CLI	Publish Single File	Scenario 2: - Pick NuGetPackageExplorer project created in scenario Common 44-47 above - Add the following lines to the csproj file

				<PublishSingleFile>true</PublishSingleFile> <RuntimeIdentifier>win-x86</RuntimeIdentifier> - Clean, Publish and Run the output. Note: "Single-File/Framework Dependent/Portable" is a valid scenario now. Scenario 3: - Pick HelloWorld project created in scenario Common 44-47 above - Clean, Run dotnet publish with an additional argument <code>/p:PublishSingleFile=true</code> <code>rd /Q /S .\bin\Release\net11.0\osx-x64</code> <code>dotnet publish -c Release -r osx-x64 /p:PublishSingleFile=true /p:IncludeNativeLibrariesForSelfExtract=true</code> <i>will only generate the single-file app and *.pdb file.</i> Run the output on a Mac machine. (Mac machine doesn't need the SDK installed). - Copy hello (the app) - run hello
				Winforms and WPF
56	PO	CLI	Winforms app	Using CLI: - Create a winforms app. Build and run. <code>dotnet new winforms -o MyWinFormsApp</code> <code>cd MyWinFormsApp</code> <code>dotnet build</code> <code>dotnet run</code> - Publish the app using - - Self contained publishing. dotnet publish -r win-x64 --self-contained - Fx dependent publishing. dotnet publish --framework net11.0-windows
57	PO	VS	LUT and UT in Winforms	- Open the above Winforms project in VS and add a .NET Xunit project to it. - Verify that LUT is working on it. - Ensure that Test Discovery automatically triggers. - Ensure that Run All/ Run from context and Run Selected tests work. Perform the same for Debug. - Ensure that Test Indicators show up in the editor on build. - Add a .NET Nunit project to it. Repeat the above verification. - Add a .NET MSTest project to it. Repeat the above verification.
58	PO	CLI	WPF app	Using CLI: - Create a WPF app. Build and run <code>dotnet new wpf -o MyWPFApp</code> <code>cd MyWPFApp</code> <code>dotnet build</code> <code>dotnet run</code> - Publish the app using - - Self contained publishing. dotnet publish -r win-x64 --self-contained - Fx dependent publishing. dotnet publish --framework net11.0-windows
59	P1	VS and CLI	Build a WPF app referencing Unity	Copy solution \\mlangfs1\public\v-masche\RealWorldSolutions\project to your machine - Modify "Microsoft.NET.Sdk.WindowsDesktop" to "Microsoft.NET.Sdk" in test.csproj file Breaking change: WinForms and WPF apps use Microsoft.NET.Sdk - .NET Microsoft Learn - Build through CLI - Open in VS and build through VS
60	PO	VS	LUT and UT in WPF	- Open the above WPF project in VS and add a .NET Xunit project to it. - Verify that LUT is working on it. - Ensure that Test Discovery automatically triggers. - Ensure that Run All/ Run from context and Run Selected tests work. Perform the same for Debug. - Ensure that Test Indicators show up in the editor on build. - Add a .NET Nunit project to it. Repeat the above verification. - Add a .NET MSTest project to it. Repeat the above verification.
				Non-native SDK support (CLIs only)
61	PO	CLI	Non-native SDK can work	Install Non-native SDK - If testing X64 OS, install X86 SDK on X64 OS - If testing Arm64 OS, - Install X64 SDK on Arm64 OS - Install X86 SDK on Arm64 OS Make sure below work. Build using Non-native SDK. Take X86 SDK for example, build using <code>C:\Program Files (x86)\dotnet\dot</code>

1. Create new console app, build and run
2. Create new WPF and Winforms apps, build and run
3. Create new Xunit, Nunit, Mstest projects. Build and run
4. Create new F# project. Build and run
5. Create new VB console app. Build and run

Note: 1.Before test x86 SDK, please enable CI, refer to steps of case 1(SDK with WiX signing :)

2.Only candidate builds for upcoming servicing releases and official previews will be signed properly to test this

Local Tools

62 P2 CLI Install and run

Using tool [NuGet Gallery | dotnet-ef 10.0.0](#)

1. Create empty directory and cd to the directory
2. Run `dotnet new tool-manifest`

File dotnet-tool.json is created under .config

```
{
  "version": 1,
  "isRoot": true,
  "tools": {}
}
```

1. Run `dotnet tool install dotnet-ef --version 10.0.0`(please choose the latest version for dotnet-ef tool from [NuGet Galle](#)

Note: please specify version when testing 10.0. otherwise, it will install default to 9.0.x if not specifying version

```
C:\Users\v-danche\tooltest>dotnet tool install dotnet-ef
You can invoke the tool from this directory using the following commands: 'dotnet tool run dotnet-ef' or 'dotnet dotnet-ef'.
Tool 'dotnet-ef' (version '7.0.10') was successfully installed. Entry is added to the manifest file C:\Users\v-danche\tooltest\.config\dotnet-tools.json.
```

2. **dotnet-tools.json should be edited and has dotnet-ef content**
3. Run `dotnet tool run dotnet-ef`

It shows ascii art and help context

```
C:\Users\v-danche\tooltest>dotnet tool run dotnet-ef

Entity Framework Core .NET Command-line Tools 7.0.10

Usage: dotnet ef [options] [command]

Options:
  --version          Show version information
  -h|--help          Show help information
  -v|--verbose       Show verbose output.
  --no-color         Don't colorize output.
  --prefix-output    Prefix output with level.

Commands:
  database           Commands to manage the database.
  dbcontext          Commands to manage DbContext types.
  migrations         Commands to manage migrations.

Use "dotnet ef [command] --help" for more information about a command.
```

4. Create a new project and run `dotnet ef dbcontext list`

1. If package Microsoft.EntityFrameworkCore.Design isn't in this project, it will return error -Your 'yourprojectname' doesn't reference Microsoft.EntityFrameworkCore.Design. having a C# projec example dotnet new console, looking for message like this - Package unity usage not found.

```
C:\Users\v-danche\tooltest>dotnet ef dbcontext list
Build started...
Build succeeded.
Your startup project 'tooltest' doesn't reference Microsoft.EntityFrameworkCore.Design. This package is required for the Entity Framework Core Tools to work. Ensure your startup project is correct, install the package, and try again.
```

2. If package Microsoft.EntityFrameworkCore.Design is in this project (run `dotnet add package Microsoft.EntityFrameworkCore.Design`) It will show Dbcontext info

```
C:\Users\v-danche\tooltest>dotnet ef dbcontext list
Build started...
Build succeeded.
No DbContext was found.
```

3. Run `dotnet tool uninstall dotnet-ef`
4. **dotnet-tools.json should go back to the original**
5. Run `dotnet tool run dotnet-ef` again
6. it should display error

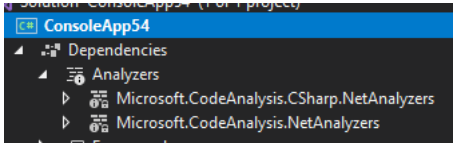
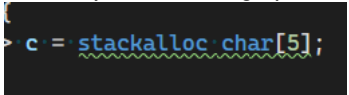
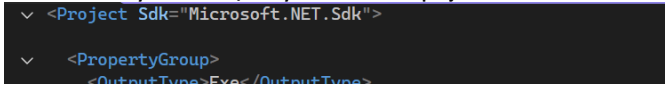
```
C:\Users\v-danche\tooltest>dotnet tool run dotnet-ef
Cannot find a tool in the manifest file that has a command named 'dotnet-ef'.
```

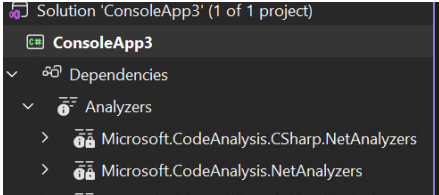
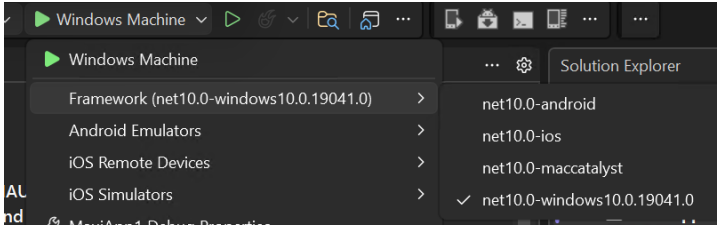
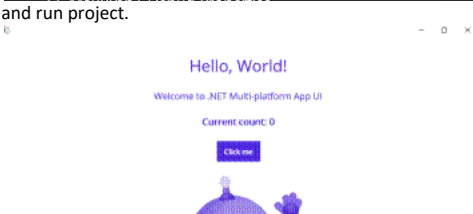
63 P2 CLI Restore and run

Using tool `dotnet-dump` [NuGet Gallery | dotnet-dump 9.0.652701](#) and [NuGet Gallery | DotNetWhy 2.1.0](#)

1. Create empty directory and cd to the directory
2. Run `dotnet new tool-manifest`
3. Copy the following to .config/dotnet-tools.json

```
{
  "version": 1,
  "isRoot": true,
  "tools": {
    "dotnet-dump": {
```

				<pre> "version": "9.0.553101", //Note:please pick up the latest version from NuGet Gallery d "commands": ["dotnet-dump"] }, "dotnetwhy": { "version": "2.1.0", //note: please pick up the latest version from NuGet Gallery D "commands": ["dotnet-why"] } } } } </pre> 4. Run <code>dotnet tool run dotnet-dump -- -h</code> 5. It should show error like below, Follow the instruction from the error. Run "dotnet tool restore" to make the "dotnet-dump" command available. 6. Run <code>dotnet tool run dotnet-dump -- -h</code> again 7. It should show help from dotnet-dump 8. Run <code>dotnet nuget locals all --clear</code> 9. Run <code>dotnet tool run dotnet-dump -- -h</code> again 10. It should show error, follow the instruction from the error Run "dotnet tool restore" to make the "dotnet-dump" command available. 11. Run <code>dotnet tool run dotnet-dump -- -h</code> again 12. It should show help from dotnet-dump
65	P1	CLI	The installers include identical contents as the zip archives	1. Installing the .NET SDK 2. Downloading and extracting the zip archive of the .NET SDK 3. Running a folder diff on the extracted SDK and the install folder to make sure that everything matches Note: - Tool WinDiff : \\mlangfs1\public\l-danche\Test\windiff.zip - It does not need to happen all the time, maybe a couple of times for each preview / release. - see the difference after uncheck 'identical files' - If the files are not identical due to the last modified date rather than FileVersion, It's normal. #9153: .NET Core SDK 3.1.111 ASP.NET Core targeting pack files are newer in .exe installer than .zip package
66	P1	VS	C++/CLI features	Test plan for C++/CLI project
				Platform Analyzers
67	P0	VS	Platform Analyzers are being imported	Install the latest SDK - Create new console app targeting .NET 11.0 or newer Verify that the following items show up under Dependencies -> Analyzers in the solution explorer  Add the following code to Program.cs <pre> using System; namespace ConsoleCore { class Program { static void Main() { while (true) { Span<char> c = stackalloc char[5]; } } } } </pre> Verify that the following expression is underlined  Verify the error list shows this warning Warning CA2014 Potential stack overflow. Move the stackalloc out of the loop. Build project, should build successfully with a single warning
68	P1	VS	Disable Platform Analyzers with analysis level	Install the latest SDK - Create new console app targeting .NET 11.0 or newer Add <AnalysisLevel>4</AnalysisLevel> to the project file like so: 

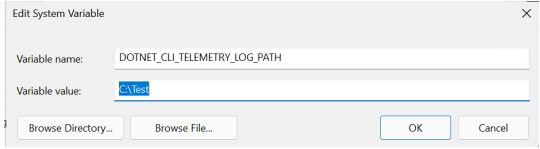
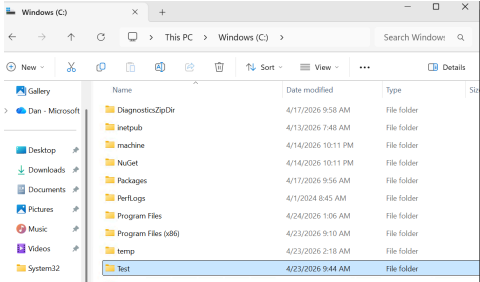
				<pre> <OutputType>Exe</OutputType> <TargetFramework>net10.0</TargetFramework> <ImplicitUsings>enable</ImplicitUsings> <Nullable>enable</Nullable> <AnalysisLevel>4</AnalysisLevel> </PropertyGroup> </Project> </pre> 3. Verify no analyzers are listed under Dependencies -> Analyzers in the solution explorer 4. Project builds without errors or warnings
69	P1	VS	Able to use warn-as-error with codeanalysis	Install the latest SDK - Create new console app targeting .NET 11.0 or newer - Verify that the following items show up under Dependencies -> Analyzers in the solution explorer  - Add the following code to Program.cs <pre> using System; namespace ConsoleCore { class Program { static void Main() { while (true) { Span<char> c = stackalloc char[5]; } } } </pre> - Add <CodeAnalysisTreatWarningsAsErrors>false</CodeAnalysisTreatWarningsAsErrors> and <TreatWarningsAsErrors>true</TreatWarningsAsErrors> to the project file like so: <pre> <Project Sdk="Microsoft.NET.Sdk"> <PropertyGroup> <OutputType>Exe</OutputType> <TargetFramework>net10.0</TargetFramework> <ImplicitUsings>enable</ImplicitUsings> <Nullable>enable</Nullable> <CodeAnalysisTreatWarningsAsErrors>false</CodeAnalysisTreatWarningsAsErrors> <TreatWarningsAsErrors>true</TreatWarningsAsErrors> </PropertyGroup> </Project> </pre> 5. Warning CA2014 shows in error list. 6. Verify project build successfully without errors but 1 warning
				.NET Maui
70	P0	VS	Validate that the latest internal build of VS is able to compile and run a .NET6 MAUI application to Reunion	.NET MAUI installation - .NET MAUI Microsoft Docs indicates the workloads that are needed to compile/run a project re - NET Multi-platform App UI development - ASP.NET and web development (required for .NET MAUI Blazor apps) - Change target framework to net11.0-windows10.0.19401 Covering maui 10.0/9.0 also  - Build and run project. 



Note: [The official .NET MAUI support policy | .NET](#)

.NET Workload

71	PO	CLI	Basic install without manifest update/workload update	- Set up clean machine with .NET SDK zip/tar.bar installation. Note: this must be a clean SDK install, you cannot run any two this scenario. - Run <code>dotnet workload install maui --skip-manifest-update --source NUGET_FEED_FROM_BUILD_EMAIL</code> Should succeed without warning or error - Run <code>dotnet workload update</code> Should succeed without warning or error - Run <code>dotnet workload list</code> Should see maui in the output matrix - Run <code>dotnet workload uninstall maui</code> The official .NET MAUI support policy .NET
72	PO	CLI	Basic workload install/list/uninstall	- Set up clean machine with .NET SDK zip/tar.bar installation. - Run <code>dotnet workload install wasm-tools --source NUGET_FEED_FROM_BUILD_EMAIL</code> Should succeed without warning or error - Run <code>dotnet workload list</code> Should see wasm-tools in the output matrix - Run <code>dotnet workload uninstall wasm-tools</code> ASP.NET Core Blazor WebAssembly build tools and ahead-of-time (AOT) compilation Microsoft Learn
73	PO		Optional workloads testing - admin installs	Scenarios are in here - test plan
				Workload sets
			Prereqs for this section	Workload sets will be published to a specific workloads release feed: https://pkgs.dev.azure.com/dnceng/public/_packaging/dotnet9-workloads/nuget/v3/index.json https://pkgs.dev.azure.com/dnceng/public/_packaging/dotnet10-workloads/nuget/v3/index.json Workload set version PR:Pipelines - Runs for dotnet-workload-versions-official-ci Runtime workloads will be in the coherency feed. Maui and Aspire workloads will be on the dotnet10 and dotnet9 feeds respectively. Finding the workload set to test Two of the three supported scenarios require having a workload set version. You can search the above workloads : update to a new workload set version, save that version off, and use that for further testing. We have plans to add t dotnet workload search in the future to make it easier What to look for Updating workload version from 8.0.400-manifests.c915c39d to <8.0.400> Successful update of workloads <code>Dotnet workload --info</code> will list the workload set version Other comments: 9.0.100-manifests.28aae763 is not an installable workload set. We decided to include a hash of the existing v specifying what workloads you had installed so we could help customers but this is not actually a workload set version flag. - The scenario for installing 8.0.303 was specific to 8.0.400. When we eventually have 9.0.2xx and 9.0.1xx vers scenario would apply (ie installing --version 9.0.101 in the 9.0.200 SDK for example). - the design spec: designs/accepted/2023/simplify-workload-versioning-implementation.md at f8990c8ce5be56a9ed08ea23a492ae5824a1d2a327 · dotnet/designs (github.com) The available workload set version can be displayed after running "dotnet workload search version" .Net 10 : [Breaking change]: Change default workload configuration from 'loose manifests' to 'workload sets' mode - Issue #4!
74	PO		Update to the latest workload set	Install maui or aspire workload(dotnet workload install maui /dotnet workload install aspire) Dotnet workload config --update-mode workload-set Dotnet workload update Check the worload version e.g dotnet workload --info /dotnet workload lis/dotnet --info/dotnet --version, you can see version Note: With .NET 10 and higher, the default mode is workload set mode now after installing workload
75	PO		Update to a specific workload set	Dotnet workload update --version <version> Tips: after 74 is done, the workload set version has been updated to the latest. - You can Switch back to manifest mode, and then go on 75p0 Dotnet workload config --update-mode manifests Dotnet workload update Or - If there is another different workload set version, you can specify it directly Or

				3. Uninstall sdk and re-install sdk, install workload and then go on 75p0
76	P0		Pin to a specific workload set	Dotnet new global.json Add "workloadVersion": "<version>" to the file Dotnet workload update Tips: after 75 is done, the workload set version has been updated to the latest. 1. You can Switch back to manifest mode Dotnet workload config --update-mode manifests Dotnet workload update and then go on 76p0 Or 2.If there is another different workload set version, you can specify it directly Or 3. Uninstall sdk and re-install sdk, install workload and then go on 76p0
77	P0		Install a workload with a specific version	dotnet workload install aspire --version <version> The Aspire workload is deprecated and no longer necessary. Aspire is now available as NuGet packages that you can add dire The official Aspire support policy .NET
78	P1		Update to a workload sets from a different band	e.g. Dotnet workload update --version 10.0.1xx (from the 10.0.2xx sdk)
79	P1		Test both admin installs and file-based	Run scenarios P0s(#74-77) with the zip install. And still run the rest P1s(#78, #80) with the exe install
80	P1		Switch back to loose manifests	Dotnet workload config --update-mode manifests Dotnet workload update
81	P2		Upgrade your SDK (if you've installed a workload set, workloads should work on upgrade across bands now)	Dotnet workload install aspire --version <version> Install an SDK from a newer feature band Dotnet new aspire
				Testing telemetry
75	P1	VS	Testing telemetry	1. Open Edit the system environment variables 2. Set DOTNET_CLI_TELEMETRY_LOG_PATH to C:\Test   2.Dotnet new console 3.Dotnet build 4.We could get the JSON file(C:\Test\Telemetry.json), and telemetry events url is /v2.1/track. <pre>{ "operationName": "System.Net.Http.HttpRequestOut", "displayName": "POST", "duration": "00:00:00.1595561", "identifiers": { "id": "00-efcc122f1291ac15c63668da1c0216b1-e7cedfe9cea732c-03", "traceId": "efcc122f1291ac15c63668da1c0216b1", "spanId": "e7cedfe9cea732c", "parentId": "00-efcc122f1291ac15c63668da1c0216b1-e890bb4cb6310ff6-00", "rootId": "efcc122f1291ac15c63668da1c0216b1", "source": { "name": "System.Net.Http", "version": "1.1", "tags": { "http.request.method": "POST", "server.address": "dc.services.visualstudio.com", "url.full": "https://dc.services.visualstudio.com/v2.1/track" } } }, "tags": { "http.request.method": "POST", "server.address": "dc.services.visualstudio.com", "url.full": "https://dc.services.visualstudio.com/v2.1/track", "network.protocol.version": "1.1", "MS.ProcessedByMetricExtractors": { (Name: X, Ver: \u00271.1\u0027) }, "events": [] }, { "operationName": "System.Net.Http.HttpRequestIn", "displayName": "POST", "duration": "00:00:00.0854072", "identifiers": { "id": "00-efcc122f1291ac15c63668da1c0216b1-d6563a9521176d66-03", "traceId": "efcc122f1291ac15c63668da1c0216b1", "spanId": "d6563a9521176d66", "parentId": "00-efcc122f1291ac15c63668da1c0216b1-e890bb4cb6310ff6-00", "rootId": "efcc122f1291ac15c63668da1c0216b1", "source": { "name": "System.Net.Http", "version": "1.1", "tags": { "http.request.method": "POST", "server.address": "southcentralus-0.in.applicationinsights.azure.com/v2.1/track", "network.protocol.version": "1.1", "MS.ProcessedByMetricExtractors": { (Name: X, Ver: \u00271.1\u0027) }, "events": [] }, { "operationName": "host-startup", "displayName": "host-startup", "duration": "00:00:00.0017585", "identifiers": { "id": "00-e7a394c094771d9f6a7621ee7abfcd0-e87305986c6f6cbc-03", "traceId": "e7a394c094771d9f6a7621ee7abfcd0", "spanId": "a87305986c6f6cbc", "parentId": "00-e7a394c094771d9f6a7621ee7abfcd0-e669bbc78769e802-00", "rootId": "e7a394c094771d9f6a7621ee7abfcd0", "source": { "name": "dotnet", "version": "11.0.100-preview.4.26215.121", "tags": { "dotnet.globalJson": "not_found", "MS.ProcessedByMetricExtractors": { (Name: X, Ver: \u00271.1\u0027) }, "events": [] } } }] }</pre>

